CPMS04

Britt Anderson

Winter 2025

1 Neural Networks

Our goal here is to understand the basic mathematical objects and their notation that underlie neural networks. We want to have enough of a grasp to be able to code simple versions of them by hand. If you ever build a big model you will undoubtedly want to use the well established standard libraries, but for making design decisions, thinking about aberrant behavior, or even just conceiving the kind of network you want to build an how it will match your problem, you want a more fundamental and practical grasp of the pieces you will be putting together.

Our distal goal is to code the Hopfield Network, and ideally the Kohonen self-organizing map, but along the way we will first learn about vectors, matrices, scalar products, the perceptron, and learning rules.

1.1 John Hopfield

To motivate our learning, and to put a human face on the project these two links give you a glimpse of Hopfield talking about his work at the 2024 Nobel Prize lectures, and a nice long podcast that gives a slow, throurough, tutorial discussion of what it is all about. We will come back to this later, but for now start listening. There is an associated homework.

1.1.1 Nobel Prize

John Hopfield's Nobel Lecture

1.1.2 Podcast on the Hopfield Network

This theoretical neuroscience podcast gives a nice long look at the Hopfield Network. It works up to its creation, its abstraction, and it social impact on

shaping the field of computational neuroscience. It is long, but, if you make your way through it you will have a good overview of the model before you start coding it.

1.2 The Math That Underlies Neural Networks?

1.2.1 Linear Algebra

The math at the heart of neural networks and their computer implementation is **linear algebra**. For us, the section of linear algebra we are going to need is mostly limited to vectors, matrices and how to add and multiply them.

Class Discussion

What makes linear algebra linear?

1. Important Objects and Operations
   - Vectors
   - Matrices
   - Scalars
   - Addition
   - Multiplication (scalar and matrix)
   - Transposition
   - Inverse

Further Class Discussion

Give a definition or description of each of these objects and operations.

```
def matA : Array (Array Int8) := #[#[1,2],#[3,4]]

def matB : Array (Array Int8) := #[#[3,4],#[1,2]]

def addMat (m n : Array (Array Int8)) := m.zipWith n (fun rm rn => rm.zipWith rn (

#eval addMat matA matB
```

1.2.2 Adding Matrices

To gain some hands on familiarity with the manipulation of matrices and vectors we will try to do some hand and programming exercises for some of the fundamental operations of addition and multiplication. We will also thereby learn that some of the rules we learned for numbers (such as a * b = b * a) do not always apply in other mathematical realms.

There are in fact many ways to think about what a vector is.

It can be thought of as a column (or row of numbers). More abstractly it is an object (arrow) with magnitude and direction. Most abstractly it is anything that obeys the requirements of a vector space.

For particular circumstances one or another of the different definitions may serve our purposes better. In application to neural networks we often just use the first definition, a column of numbers, but the second can be more helpful for developing our geometric intuitions about what various learning rules are doing and how they do it.

Similarly, we often just consider a matrix as a collection of vectors or as a rectangular (2-D) collection of numbers.

1.2.3 Activity

In your programming language of choice find the package (sometimes also called a library) that has the construct of a "vector". Is it the same thing as the vector described above? If not, make sure you find the equivalent construct for your programming language. Make sure you can use it to create two vectors and computer their scalar product. Be prepared to demonstrate this for the class.

```
(require math/matrix)
(define a-mat (matrix [[1 2 3]]))
(define b-mat (matrix [[4 5 6]]))
;adding two vectors
(matrix+ a-mat b-mat)
(matrix* a-mat (matrix-transpose b-mat))
```

Figure 1: This demonstrates the idea for the racket programming language where the library for dealing with vector like things is called `math/matrix`.

Important: many programming languages have structures called vectors that refer to their organization in memory, not their mathematical character. Don't just assumed that "vector" is what you are looking for.

Further Class Discussion

Make two arrays and make them the same size. Then add them. What is the *size* of a matrix?

```
def matA : Array (Array Int8) := #[#[1,2],#[3,4]]

def matB : Array (Array Int8) := #[#[3,4],#[1,2]]

def addMat (m n : Array (Array Int8)) :=
m.zipWith n (fun rm rn => rm.zipWith rn
(fun a b => a + b))

#eval addMat matA matB
```

Figure 2: My effort to do this in lean4.

Do the same for matrix multiplication. Note that there are particular requirements for the sizes of matrices in order that it is possible to multiply them in both directions. What is that rule?

Class Discussion

What is the name for the property of having A*B = B*A?

1.2.4 Common Notational Conventions for Vectors and Matrices

Vectors tend to be notated as *lower case* letters, often in bold, such as $\mathbf{a}$. They are also occasionally represented with little arrows on top such as $\vec{\mathbf{a}}$.

Matrices tend to be notated as *upper case* letters, typically in bold, such as $\mathbf{M}$.

Good things to know: what is an inner product? How does it differ from a scalar product. Can you write code in your preferred language for the inner product?

1.3 What is a Neural Network?

What is a Neural Network? It is a brain inspired computational approach in which "neurons" compute functions of their inputs and pass on a *weighted* proportion to the next neuron in the chain.

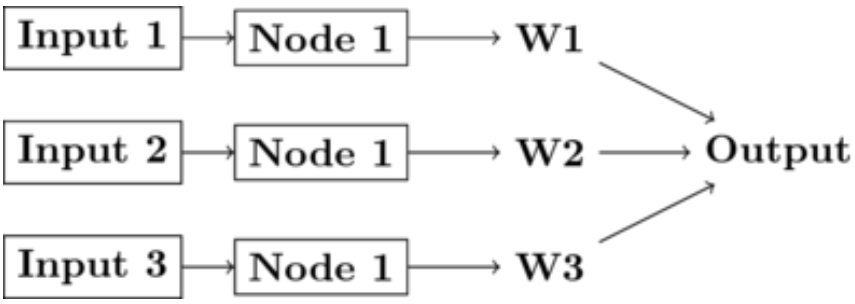

Figure 3: Simple schematic of the basics of a neural network. This is an image for a single neuron. The input has three elements and each of these connects to the same neuron ("node 1"). The activity at those nodes is filtered by the weights, which are specific for each of the inputs. These three processed inputs are combined to generate the output from this neuron. For multiple layers this output becomes an input for the next neuron along the chain.

1.3.1 Non-linearities

The spiking of a biological neuron is non-linear. You will see this in both the integrate and fire and Hodgkin and Huxley models you're going to program. For our neural networks we will usually use a *threshold*. When the threshold exceeds a certain value our activity will jump (or step) to a new value.

$$\text{if } I_1 \times w_{1,1} + I_2 \times w_{2,1} + I_3 \times w_{3,1} > \Theta \text{ then } Output = 1 \tag{1}$$

What this equation shows is that Inputs ("I"'s) are passed to a neuron (also called a node or sometimes unit). Those inputs have something like a synapse. That is designated by the w's for weights. Those weights are how tightly the input and internal activity of our artificial neuron are coupled. The reason for all the subscripts is to try and help you see the similarity between this equation and the inner product and matrix multiplication rules you just worked on programming. The activity of the neuron is a sort of internal state, and then, based on the comparison of that activity to the threshold, you can envision the neuron spiking or not, meaning it has value 1 or 0. Mathematically, the weighted sum is fed into a threshold function that compares the value to a threshold (Θ), and passes on the value 1 if it is greater than the threshold and 0 (sometimes -1) for the inactive state.

To prepare you for the next steps in writing a simple perceptron (the earliest form of artificial neural network), you should try to answer the following

questions.

Class Discussion

Questions:

- What, geometrically speaking, is a plane?
- What is a hyperplane?
- What is linearly separability and how does that relate to planes and hyperplanes?

One of our first efforts will be to code a @italic{perceptron} to solve the XOR problem. In order for this to happen you need to know a bit about @italic{Boolean} functions and what an XOR problem actually is.

@bold{Examples of Boolean Functions and How They Map onto our Neural Network Intuitions}

1.3.2 Exercise XOR}

1.3.3 Connections}

1.3.4 Boolean Logic}

1.3.5 First Order Logic - Truth Tables}

1.3.6 How does a network like this work?}

1.3.7 Test your understanding:}

1.3.8 A Worked Example}

1.3.9 Distance Metrics}

1.3.10 Hebb's @hyperlink["https://en.wikipedia.org/wiki/Outer_product"]{Outer Product} Rule}